

SECTION 3: Large Language Models (LLMs)

In this section, we dive into the core technology behind ChatGPT, Claude, and Gemini. We explore the evolution from simple statistical models to modern Large Language Models (LLMs), the breakthrough Transformer architecture, how these models are trained, and exactly how they process and generate human-like text.

Lecture 3.1: Introduction to Language Models

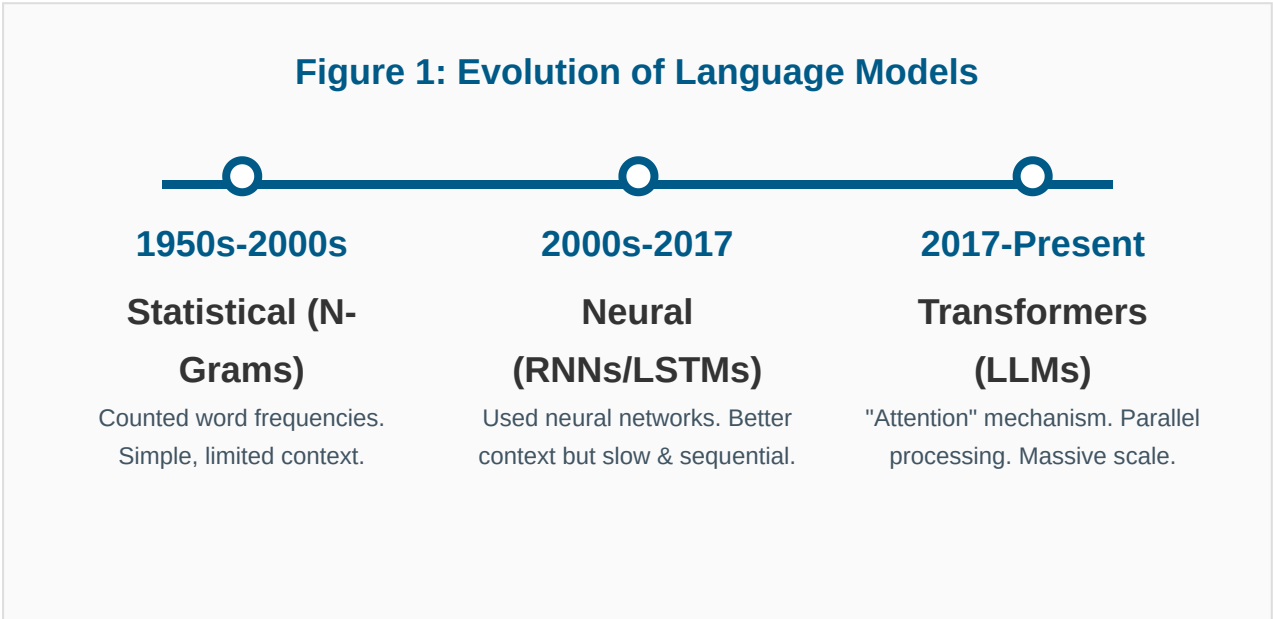
What is a Language Model?

At its core, a **Language Model** is a statistical tool that predicts the next word (or sequence of words) in a sentence based on context. It assigns probabilities to sequences of words.

Analogy: The Brain's Prediction

If someone says "The cat sat on the...", your brain automatically predicts "mat", "floor", or "chair". It assigns a near-zero probability to "elephant". LLMs do this mathematically on a massive scale.

The Evolution of Language Models



What Makes an LLM "Large"?

1. **Massive Parameters:** Billions or trillions of adjustable weights (GPT-3 has 175B).
2. **Massive Data:** Trained on petabytes of text (books, web, code).
3. **Massive Compute:** Requires thousands of GPUs and millions of dollars to train.

Capabilities: Beyond Text Generation

While trained only to predict the next word, LLMs develop **emergent capabilities**—skills they weren't explicitly taught but learned as a byproduct of understanding language structure.

- **Text Generation:** Writing poems, essays, emails.
- **Summarization:** Condensing long articles.
- **Translation:** Converting languages.
- **Code Generation:** Writing functional programming code.
- **Reasoning:** Solving logic puzzles and multi-step problems.

LECTURE 3.1 KEY TAKEAWAY

LLMs are advanced statistical predictors that learn grammar, semantics, and world knowledge by reading massive amounts of text. Their ability to perform diverse tasks (coding, translation, reasoning) emerges from simply learning to predict the next word very well.

Lecture 3.2: Transformers Architecture Explained

The Breakthrough: "Attention Is All You Need"

In 2017, Google researchers published this landmark paper introducing the **Transformer**. It solved the critical bottlenecks of previous RNN models:

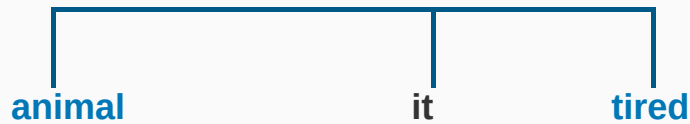
- **Parallel Processing:** Can process all words in a sentence simultaneously (unlike RNNs).
- **Long-Range Dependencies:** Can connect words far apart in a text (e.g., relating a pronoun to a name mentioned paragraphs ago).

Self-Attention Mechanism

Self-attention allows the model to look at other words in the sentence to understand the context of the current word.

Figure 2: Self-Attention Example

*"The animal didn't cross the street because **it** was too tired."*



To understand what "it" refers to, the model "attends" strongly to "animal" and "tired".

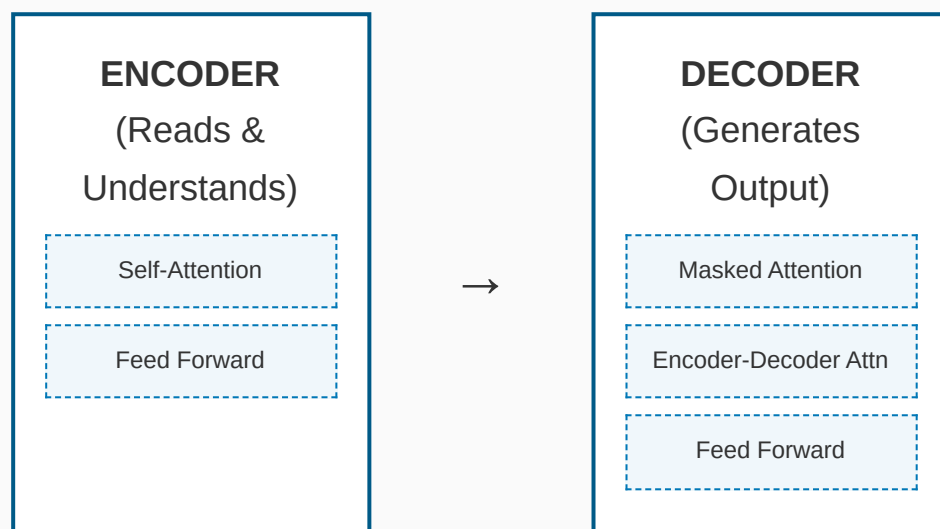
Key Components

Query, Key, and Value (Q, K, V)

Think of it like a database lookup:

- **Query:** What I'm looking for.
- **Key:** The label I match against.
- **Value:** The actual information I retrieve.

Figure 3: Transformer Architecture Simplified



Modern GPT models use a **Decoder-Only** architecture.

LECTURE 3.2 KEY TAKEAWAY

Transformers revolutionized AI by allowing parallel processing and using "Self-Attention" to understand the relationships between all words in a sentence simultaneously, enabling deep contextual understanding.

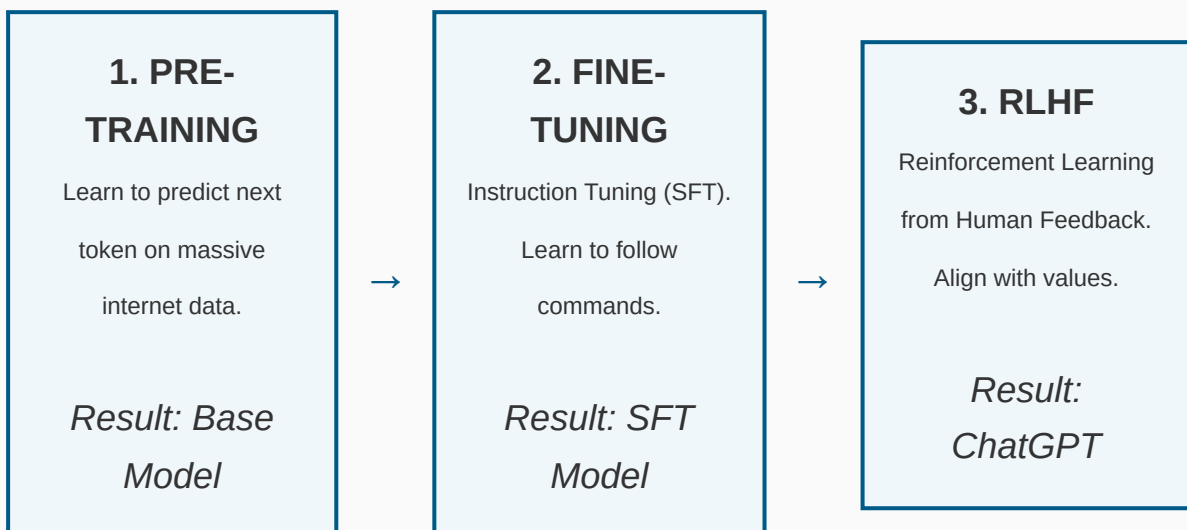
Lecture 3.3: How ChatGPT and GPT Models Work

GPT: Generative Pre-trained Transformer

- **Generative:** Creates new data.
- **Pre-trained:** Learns general knowledge first.
- **Transformer:** Uses the architecture we just learned.

The Training Pipeline

Figure 4: How ChatGPT is Made



How It Generates Text (Autoregressive)

GPT generates text one token at a time. It takes your prompt, predicts the most likely next token, outputs it, and then feeds that output back into itself to predict the *next* one.

Controlling the Output

- **Temperature:** Controls randomness. Low (0.2) = Focused/Deterministic. High (0.8) = Creative/Random.

- **Context Window:** The limit on how much text the model can "remember" at once (Prompt + Response).

Model Evolution

Model	Parameters	Year	Key Feature
GPT-1	117 Million	2018	Proof of concept
GPT-2	1.5 Billion	2019	Coherent text generation
GPT-3	175 Billion	2020	General purpose capabilities
GPT-4	Trillions (est.)	2023	Reasoning, coding, multimodal

LECTURE 3.3 KEY TAKEAWAY

ChatGPT is a fine-tuned GPT model. It goes through massive pre-training to learn language, then instruction tuning and RLHF to become a helpful, safe assistant. It generates text sequentially, one piece at a time.

Lecture 3.4: Tokenization and Text Processing

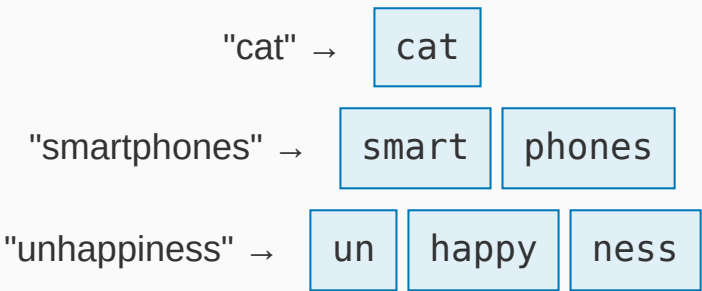
The Problem: Computers Can't Read Words

Neural networks only understand numbers. We must convert text into numbers. Simple word-level conversion (Cat = 1, Dog = 2) is inefficient because of the sheer number of words in English.

The Solution: Subword Tokenization (BPE)

Modern LLMs use **Byte Pair Encoding (BPE)**. It breaks words into smaller meaningful chunks (tokens). Common words are single tokens; rare words are broken down.

Figure 5: Tokenization Examples



Why Tokenization Matters

- **Cost:** APIs charge per token (approx 0.75 words = 1 token).
- **Limits:** Context windows are measured in tokens.
- **Performance:** Bad tokenization leads to poor understanding (e.g., spelling tasks).

The Spelling Blind Spot

Why can't LLMs spell "Lollipop" backwards easily?

Because to the model, "Lollipop" is just **Token ID: 34591**. It doesn't "see" the letters L-o-l-l-i-p-o-p unless it breaks it down character by character.

LECTURE 3.4 KEY TAKEAWAY

Text is chopped into tokens (sub-words) before entering the model. This is efficient but causes quirks in how models handle spelling and math. Always think in tokens, not words, when calculating costs or limits.

Lecture 3.5: Embeddings and Vector Representations

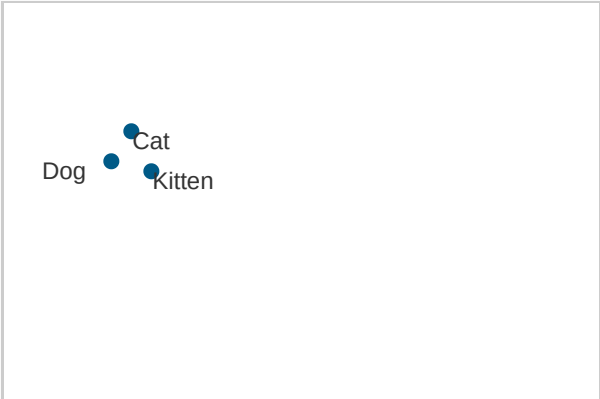
From ID to Meaning

Token IDs (e.g., 34591) are just arbitrary numbers. To understand *meaning*, models map these tokens to **Embeddings**—lists of numbers (vectors) that represent the semantic meaning.

Semantic Space

In this vector space, words with similar meanings are mathematically close to each other.

Figure 6: 2D Embedding Visualization



Dog • Cat
• Kitten



Chair

Table

Vector Arithmetic

We can do math with meaning because of embeddings.

$$\text{King} - \text{Man} + \text{Woman} \approx \text{Queen}$$

If you take the vector for "King", subtract the "Man" concept, and add the "Woman" concept, the resulting vector is closest to "Queen".

Contextual Embeddings

Unlike older static embeddings (Word2Vec), Transformer embeddings are **contextual**.

- "I went to the **bank** to deposit money." (Financial embedding)
- "I sat on the river **bank**." (Geographical embedding)

The model adjusts the vector based on the surrounding words.

LECTURE 3.5 KEY TAKEAWAY

Embeddings translate words into "concepts" (lists of numbers). Similar concepts are close together in this mathematical space. This allows the model to understand analogies, relationships, and context.

Lecture 3.6: The Generation Process

Autoregressive Generation

The model writes like a human typing: one word at a time, looking back at what was just written.

Figure 7: Step-by-Step Generation

Input: "The future of AI is" → Model predicts → "bright"

Input: "The future of AI is bright" → Model predicts → "and"

Input: "The future of AI is bright and"

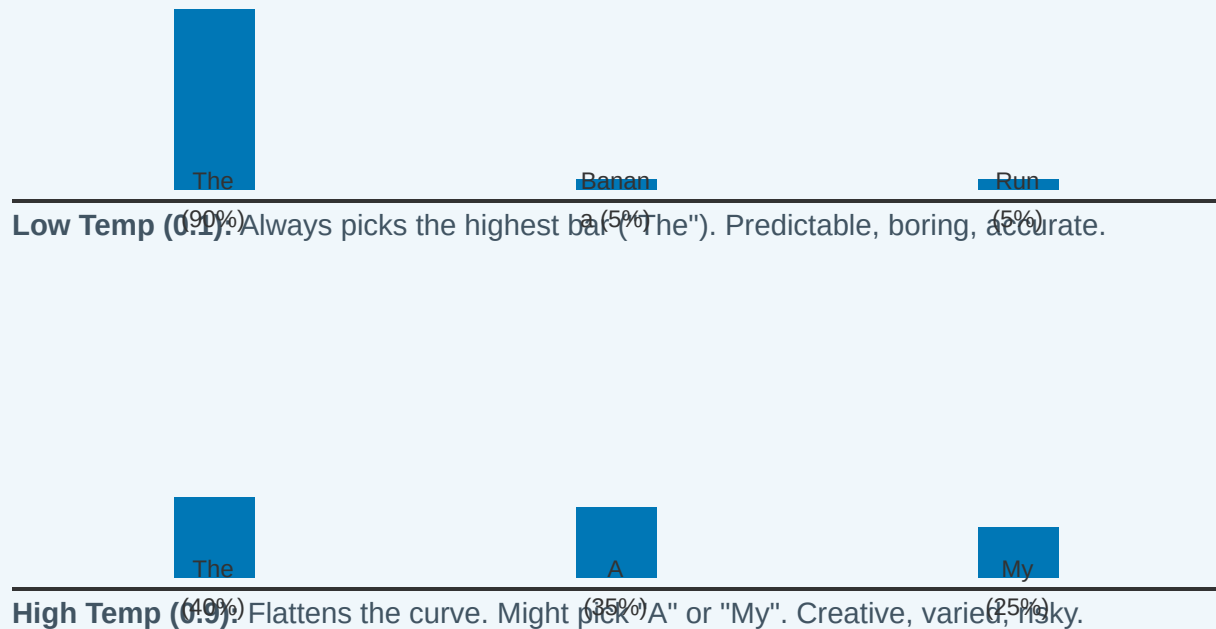
→ Model predicts →

"full"

Sampling Strategies: How it Picks the Word

The model outputs a probability list for the next word. How do we choose?

The Temperature Dial



- **Greedy Decoding:** Always pick #1. Safe but repetitive.
- **Top-K Sampling:** Only consider top K words (e.g., top 10).
- **Top-P (Nucleus) Sampling:** Consider words adding up to P% probability (e.g., top 90%). Dynamic and popular.

LECTURE 3.6 KEY TAKEAWAY

Models generate text sequentially (autoregressively). They don't plan the whole sentence ahead. Strategies like Temperature and Top-P sampling balance predictability with creativity.